

Exercise 1: Configuring a Basic Spring Application

Scenario: Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.

Steps:

1. Set Up a Spring Project:

- Create a Maven project named **LibraryManagement**.
- Add Spring Core dependencies in the **pom.xml** file.

2. Configure the Application Context:

- Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
- Define beans for **BookService** and **BookRepository** in the XML file.

3. Define Service and Repository Classes:

- Create a package **com.library.service** and add a class **BookService**.
- Create a package **com.library.repository** and add a class **BookRepository**.

4. Run the Application:

- Create a main class to load the Spring context and test the configuration.

Solution:

1. Set Up a Spring Project:

1. Maven project : “ maven-archetype-quickstart “ (Version 1.5)

New Maven Project

Specify Archetype parameters

Group Id:

Artifact Id:

Version:

Package:

☒ run archetype generation interactively

Properties available from archetype:

Name	Value
javaCompilerVersion	17
junitVersion	5.11.0

Advanced

< Back Next > Finish Cancel

2. pom.xml file – add dependency

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>6.1.14</version>
</dependency>
```

2. Configure the Application Context:

1. applicationContext.xml – created resources folder in the src/main/java

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="bookRepository"
    class="com.library.repository.BookRepository"/>
  <bean id="bookService"
    class="com.library.service.BookService">
    <property name="bookRepository"
      ref="bookRepository"/>
  </bean>
</beans>
```

3. Define Service and Repository Classes:

1. BookService.java – created and saved in the respected package

```
package com.library.service;
import com.library.repository.BookRepository;
public class BookService {
  private BookRepository bookRepository;
  public void setBookRepository(BookRepository bookRepository) {
    this.bookRepository = bookRepository;
  }
  public void showBooks() {
    System.out.println("Book Service Called for upendra...");
    bookRepository.displayBooks();
  }
}
```

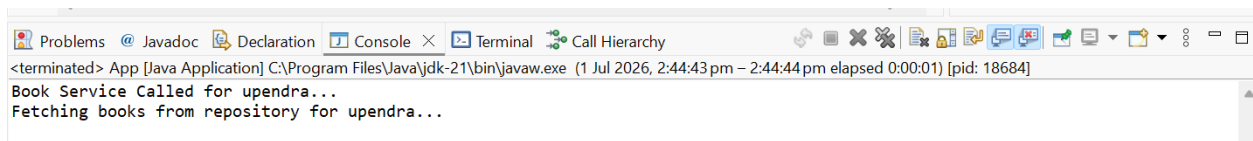
2. BookRepository.java – created and saved in the respective package

```
package com.library.repository;
public class BookRepository {
  public void displayBooks() {
    System.out.println("Fetching books from repository for upendra...");
  }
}
```

4. Run the Application:

```
package com.library;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;
public class App {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext(
                "applicationContext.xml"
            );
        BookService service =
            context.getBean(
                "bookService",
                BookService.class
            );
        service.showBooks();
    }
}
```

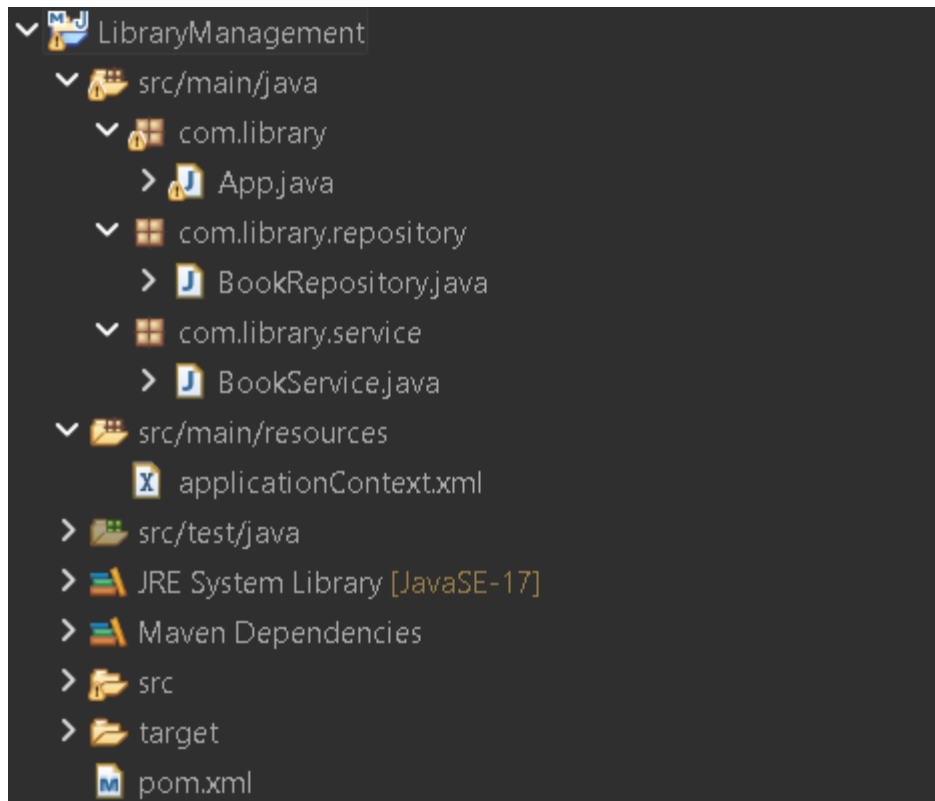
Output:



The screenshot shows an IDE console window with the following output:

```
<terminated> App [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (1 Jul 2026, 2:44:43 pm - 2:44:44 pm elapsed 0:00:01) [pid: 18684]
Book Service Called for upendra...
Fetching books from repository for upendra...
```

Project :



Exercise 2: Implementing Dependency Injection

Scenario:

In the library management application, you need to manage the dependencies between the **BookService** and **BookRepository** classes using Spring's IoC and DI.

Steps:

1. **Modify the XML Configuration:**
 - Update **applicationContext.xml** to wire **BookRepository** into **BookService**.
2. **Update the BookService Class:**
 - Ensure that **BookService** class has a setter method for **BookRepository**.
3. **Test the Configuration:**
 - Run the **LibraryManagementApplication** main class to verify the dependency injection.

Solution:

1. **Modify the XML Configuration**

Added in the bookService bean tags

```
<property name="bookRepository" ref="bookRepository"/>
```

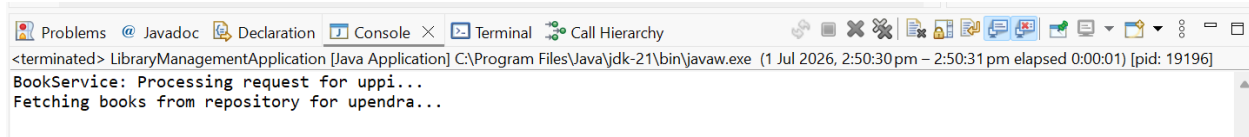
2. Update the BookService Class

```
package com.library.service;
import com.library.repository.BookRepository;
public class BookService {
    private BookRepository bookRepository;
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }
    public void issueBook() {
        System.out.println("BookService: Processing request uppi...");
        bookRepository.displayBooks();
    }
}
```

3. Test the Configuration

```
package com.library;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;
public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext(
                "applicationContext.xml"
            );
        BookService service =
            context.getBean(
                "bookService",
                BookService.class
            );
        service.issueBook();
    }
}
```

Output:



Exercise 4: Creating and Configuring a Maven Project

Scenario:

You need to set up a new Maven project for the library management application and add Spring dependencies.

Steps:

1. Create a New Maven Project:

- Create a new Maven project named **LibraryManagement**.

2. Add Spring Dependencies in pom.xml:

- Include dependencies for Spring Context, Spring AOP, and Spring WebMVC.

3. Configure Maven Plugins:

- Configure the Maven Compiler Plugin for Java version 1.8 in the pom.xml file.

Solution:

```
<dependencies>
  <!-- Spring Context -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>${spring.version}</version>
  </dependency>
  <!-- Spring AOP -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aop</artifactId>
    <version>${spring.version}</version>
  </dependency>
  <!-- Spring Web MVC -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>${spring.version}</version>
  </dependency>
</dependencies>
```

1. Added Pulgin

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>
    maven-compiler-plugin
  </artifactId>
  <version>3.13.0</version>
  <configuration>
    <source>1.8</source>
    <target>1.8</target>
  </configuration>
</plugin>
```